

Distance computations (`scipy.spatial.distance`)

Function Reference

Distance matrix computation from a collection of raw observation vectors stored in a rectangular array.

<i>Function</i>	<i>Description</i>
<code>pdist</code>	pairwise distances between observation vectors.
<code>cdist</code>	distances between two collections of observation vectors.
<code>squareform</code>	converts a square distance matrix to a condensed one and vice versa.

Predicates for checking the validity of distance matrices, both condensed and redundant. Also contained in computing the number of observations in a distance matrix.

<i>Function</i>	<i>Description</i>
<code>is_valid_dm</code>	checks for a valid distance matrix.
<code>is_valid_y</code>	checks for a valid condensed distance matrix.
<code>num_obs_dm</code>	# of observations in a distance matrix.
<code>num_obs_y</code>	# of observations in a condensed distance matrix.

Distance functions between two vectors u and v . Computing distances over a large collection of vectors is inefficient for this purpose.

<i>Function</i>	<i>Description</i>
<code>braycurtis</code>	the Bray-Curtis distance.
<code>canberra</code>	the Canberra distance.
<code>chebyshev</code>	the Chebyshev distance.
<code>cityblock</code>	the Manhattan distance.
<code>correlation</code>	the Correlation distance.
<code>cosine</code>	the Cosine distance.
<code>dice</code>	the Dice dissimilarity (boolean).
<code>euclidean</code>	the Euclidean distance.
<code>hamming</code>	the Hamming distance (boolean).
<code>jaccard</code>	the Jaccard distance (boolean).
<code>kulsinski</code>	the Kulsinski distance (boolean).
<code>mahalanobis</code>	the Mahalanobis distance.
<code>matching</code>	the matching dissimilarity (boolean).
<code>minkowski</code>	the Minkowski distance.
<code>rogerstanimoto</code>	the Rogers-Tanimoto dissimilarity (boolean).
<code>russellrao</code>	the Russell-Rao dissimilarity (boolean).
<code>seuclidean</code>	the normalized Euclidean distance.
<code>sokalmichener</code>	the Sokal-Michener dissimilarity (boolean).
<code>sokalsneath</code>	the Sokal-Sneath dissimilarity (boolean).
<code>squeuclidean</code>	the squared Euclidean distance.
<code>yule</code>	the Yule dissimilarity (boolean).

References

- [Sta07] "Statistics toolbox." API Reference Documentation. The MathWorks. <http://www.mathworks.com/access/helpdesk/help/toolbox/stats/>. Accessed October 1, 2007.
- [Mti07] "Hierarchical clustering." API Reference Documentation. The Wolfram Research, Inc. <http://reference.wolfram.com/mathematica/HierarchicalClustering/tutorial/HierarchicalClustering.html>.
- [Gow69] Gower, JC and Ross, GJS. "Minimum Spanning Trees and Single Linkage Cluster Analysis." *Applied* 1969.
- [War63] Ward Jr, JH. "Hierarchical grouping to optimize an objective function." *Journal of the American Statistical Association* pp. 236–44. 1963.
- [Joh66] Johnson, SC. "Hierarchical clustering schemes." *Psychometrika*. 32(2): pp. 241–54. 1966.
- [Sne62] Sneath, PH and Sokal, RR. "Numerical taxonomy." *Nature*. 193: pp. 855–60. 1962.
- [Bat95] Batagelj, V. "Comparing resemblance measures." *Journal of Classification*. 12: pp. 73–90. 1995.
- [Sok58] Sokal, RR and Michener, CD. "A statistical method for evaluating systematic relationships." *Scientific* 38. 1958.
- [Ede79] Edelbrock, C. "Mixture model tests of hierarchical clustering algorithms: the problem of classifying events." *Behavioral Research*. 14: pp. 367–84. 1979.
- [Jai88] Jain, A., and Dubes, R., "Algorithms for Clustering Data." Prentice-Hall. Englewood Cliffs, NJ. 1988.
- [Fis36] Fisher, RA "The use of multiple measurements in taxonomic problems." *Annals of Eugenics*, 7(2): 179–

Copyright Notice

Copyright (C) Damian Eads, 2007-2008. New BSD License.

`scipy.spatial.distance.braycurtis(u, v)`

Computes the Bray-Curtis distance between two n -vectors u and v , which is defined as

$$\sum |u_i - v_i| / \sum |u_i + v_i|.$$

Parameters:	u : ndarray An n -dimensional vector. v : ndarray An n -dimensional vector.
Returns:	d : double The Bray-Curtis distance between vectors u and v .

`scipy.spatial.distance.canberra(u, v)`

Computes the Canberra distance between two n -vectors u and v , which is defined as

$$\frac{\sum_i |u_i - v_i|}{\sum_i |u_i| + |v_i|}.$$

Parameters:	u : ndarray An n -dimensional vector. v : ndarray An n -dimensional vector.
Returns:	d : double The Canberra distance between vectors u and v .

`scipy.spatial.distance.cdist(XA, XB, metric='euclidean', p=2, V=None, VI=None, w=None)`

Computes distance between each pair of observation vectors in the Cartesian product of two collections array while x_B is a m_B by n array. A m_A by m_B array is returned. An exception is thrown if x_A and x_B do not have the same number of columns.

A rectangular distance matrix y is returned. For each i and j , the metric $\text{dist}(u=XA[i], v=XB[j])$ is computed

The following are common calling conventions:

1. $Y = \text{cdist}(XA, XB, \text{'euclidean'})$

Computes the distance between m points using Euclidean distance (2-norm) as the distance metric b are arranged as m n -dimensional row vectors in the matrix X .

2. $Y = \text{cdist}(XA, XB, \text{'minkowski'}, p)$

Computes the distances using the Minkowski distance $\|u - v\|_p$ (p -norm) where $p \geq 1$.

3. $Y = \text{cdist}(XA, XB, \text{'cityblock'})$

Computes the city block or Manhattan distance between the points.

4. $Y = \text{cdist}(XA, XB, \text{'seuclidean'}, V=None)$

Computes the standardized Euclidean distance. The standardized Euclidean distance between two n -

$$\sqrt{\sum (u_i - v_i)^2 / V[x_i]}.$$

V is the variance vector; $V[i]$ is the variance computed over all the i 'th components of the points. If not passed, it is automatically computed.

5. $Y = \text{cdist}(XA, XB, \text{'sqeuclidean'})$

Computes the squared Euclidean distance $\|u - v\|_2^2$ between the vectors.

6. $Y = \text{cdist}(XA, XB, \text{'cosine'})$

Computes the cosine distance between vectors u and v ,

$$\frac{1 - uv^T}{|u|_2 |v|_2}$$

where $|*|_2$ is the 2-norm of its argument $*$.

7. `Y = cdist(XA, XB, 'correlation')`

Computes the correlation distance between vectors u and v . This is

$$\frac{1 - (u - n|u|_1)(v - n|v|_1)^T}{|(u - n|u|_1)|_2|(v - n|v|_1)|_2^T}$$

where $| \cdot |_1$ is the Manhattan (or 1-norm) of its argument, and n is the common dimensionality of the u and v .

8. `Y = cdist(XA, XB, 'hamming')`

Computes the normalized Hamming distance, or the proportion of those vector elements between u and v that disagree. To save memory, the matrix x can be of type boolean.

9. `Y = cdist(XA, XB, 'jaccard')`

Computes the Jaccard distance between the points. Given two vectors, u and v , the Jaccard distance is the proportion of elements $u[i]$ and $v[i]$ that disagree where at least one of them is non-zero.

10. `Y = cdist(XA, XB, 'chebyshev')`

Computes the Chebyshev distance between the points. The Chebyshev distance between two n -maximum norm-1 distance between their respective elements. More precisely, the distance is given by

$$d(u, v) = \max_i |u_i - v_i|.$$

11. `Y = cdist(XA, XB, 'canberra')`

Computes the Canberra distance between the points. The Canberra distance between two points u and v is

$$d(u, v) = \sum_i \frac{|u_i - v_i|}{(|u_i| + |v_i|)}$$

12. `Y = cdist(XA, XB, 'braycurtis')`

Computes the Bray-Curtis distance between the points. The Bray-Curtis distance between two points u and v is

$$d(u, v) = \frac{\sum_i (u_i + v_i)}{\sum_i (u_i + v_i)}$$

13. `Y = cdist(XA, XB, 'mahalanobis', VI=None)`

Computes the Mahalanobis distance between the points. The Mahalanobis distance between u and v is $\sqrt{(u - v)^T (1/V) (u - v)}$ where $(1/V)$ (the `VI` variable) is the inverse covariance. If `VI` is not `None`, it is the inverse covariance matrix.

14. `Y = cdist(XA, XB, 'yule')`

Computes the Yule distance between the boolean vectors. (see yule function documentation)

15. `Y = cdist(XA, XB, 'matching')`

Computes the matching distance between the boolean vectors. (see matching function documentation)

16. `Y = cdist(XA, XB, 'dice')`

Computes the Dice distance between the boolean vectors. (see dice function documentation)

17. `Y = cdist(XA, XB, 'kulsinski')`

Computes the Kulsinski distance between the boolean vectors. (see kulsinski function documentation)

18. `Y = cdist(XA, XB, 'rogerstanimoto')`

Computes the Rogers-Tanimoto distance between the boolean vectors. (see rogerstanimoto function documentation)

19. `Y = cdist(XA, XB, 'russellrao')`

Computes the Russell-Rao distance between the boolean vectors. (see russellrao function documentation)

20. `Y = cdist(XA, XB, 'sokalmichener')`

Computes the Sokal-Michener distance between the boolean vectors. (see sokalmichener function documentation)

21. `Y = cdist(XA, XB, 'sokalsneath')`

Computes the Sokal-Sneath distance between the vectors. (see sokalsneath function documentation)

22. `Y = cdist(XA, XB, 'wminkowski')`

Computes the weighted Minkowski distance between the vectors. (see sokalsneath function document)

23. `Y = cdist(XA, XB, f)`

Computes the distance between all pairs of vectors in X using the user supplied 2-arity function f. The distance between the vectors could be computed as follows:

```
dm = cdist(XA, XB, (lambda u, v: np.sqrt(((u-v)*(u-v).T).sum())))
```

Note that you should avoid passing a reference to one of the distance functions defined in this library.

```
dm = cdist(XA, XB, sokalsneath)
```

would calculate the pair-wise distances between the vectors in X using the Python function sokalsneath being called $\binom{n}{2}$ times, which is inefficient. Instead, the optimized C version is more efficient. The following syntax:

```
dm = cdist(XA, XB, 'sokalsneath')
```

Parameters:	XA : ndarray
	An m_A by n array of m_A original observations in an n -dimensional space.
	XB : ndarray
	An m_B by n array of m_B original observations in an n -dimensional space.
	metric : string or function
	The distance metric to use. The distance function can be 'braycurtis', 'canberra', 'correlation', 'cosine', 'dice', 'euclidean', 'hamming', 'jaccard', 'kulsinski', 'mahalanobis', 'rogerstanimoto', 'russellrao', 'seuclidean', 'sokalmichener', 'sokalsneath', 'sqeuclidean', 'tanimoto', 'yule', 'zincburys'
	w : ndarray
	The weight vector (for weighted Minkowski).
	p : double
	The p-norm to apply (for Minkowski, weighted and unweighted)
Returns:	Y : ndarray
	A m_A by m_B distance matrix.

`scipy.spatial.distance.chebyshev(u, v)`

Computes the Chebyshev distance between two n-vectors u and v, which is defined as

$$\max_i |u_i - v_i|.$$

Parameters:	u : ndarray
	An n -dimensional vector.
	v : ndarray
	An n -dimensional vector.
Returns:	d : double
	The Chebyshev distance between vectors u and v.

`scipy.spatial.distance.cityblock(u, v)`

Computes the Manhattan distance between two n-vectors u and v, which is defined as

$$\sum_i (u_i - v_i).$$

Parameters:	u : ndarray
	An n -dimensional vector.
	v : ndarray
	An n -dimensional vector.
Returns:	d : double
	The City Block distance between vectors u and v.

`scipy.spatial.distance.correlation(u, v)`

Computes the correlation distance between two n -vectors u and v , which is defined as

$$\frac{1 - (u - \bar{u})(v - \bar{v})^T}{\|u - \bar{u}\|_2 \|v - \bar{v}\|_2}$$

where $\bar{u}$ is the mean of a vectors elements and n is the common dimensionality of u and v .

Parameters:	u : ndarray An n -dimensional vector.
	v : ndarray An n -dimensional vector.
Returns:	d : double The correlation distance between vectors u and v .

`scipy.spatial.distance.cosine(u, v)`

Computes the Cosine distance between two n -vectors u and v , which is defined as

$$\frac{1 - uv^T}{\|u\|_2 \|v\|_2}$$

Parameters:	u : ndarray An n -dimensional vector.
	v : ndarray An n -dimensional vector.
Returns:	d : double The Cosine distance between vectors u and v .

`scipy.spatial.distance.dice(u, v)`

Computes the Dice dissimilarity between two boolean n -vectors u and v , which is

$$\frac{c_{TF} + c_{FT}}{2c_{TT} + c_{FT} + c_{TF}}$$

where c_{ij} is the number of occurrences of $u[k] = i$ and $v[k] = j$ for $k < n$.

Parameters:	u : ndarray An n -dimensional vector.
	v : ndarray An n -dimensional vector.
Returns:	d : double The Dice dissimilarity between vectors u and v .

`scipy.spatial.distance.euclidean(u, v)`

Computes the Euclidean distance between two n -vectors u and v , which is defined as

$$\|u - v\|_2$$

Parameters:	u : ndarray An n -dimensional vector.
	v : ndarray An n -dimensional vector.
Returns:	d : double The Euclidean distance between vectors u and v .

`scipy.spatial.distance.hamming(u, v)`

Computes the Hamming distance between two n -vectors u and v , which is simply the proportion of disagree u and v are boolean vectors, the Hamming distance is

$$\frac{c_{01} + c_{10}}{n}$$

where c_{ij} is the number of occurrences of $u[k] = i$ and $v[k] = j$ for $k < n$.

Parameters: u : ndarray
An n -dimensional vector.
 v : ndarray
An n -dimensional vector.

Returns: d : double
The Hamming distance between vectors u and v .

`scipy.spatial.distance.is_valid_dm(D, tol=0.0, throw=False, name='D', warning=False)`

Returns True if the variable D passed is a valid distance matrix. Distance matrices must be 2-dimensional doubles. They must have a zero-diagonal, and they must be symmetric.

Parameters: D : ndarray
The candidate object to test for validity.
 tol : double
The distance matrix should be symmetric. tol is the maximum difference between the i th entry for the distance metric to be considered symmetric.
 $throw$: bool
An exception is thrown if the distance matrix passed is not valid.
 $name$: string
the name of the variable to be checked. This is useful if $throw$ is set to `True` so the identified in the exception message when an exception is thrown.
 $warning$: bool
Instead of throwing an exception, a warning message is raised.

Returns: Returns `True` if the variable D passed is a valid distance matrix. Small numerical differences of the diagonal are ignored if they are within the tolerance specified by tol .

`scipy.spatial.distance.is_valid_y(y, warning=False, throw=False, name=None)`

Returns `True` if the variable y passed is a valid condensed distance matrix. Condensed distance matrices are arrays containing doubles. Their length must be a binomial coefficient $\binom{n}{2}$ for some positive integer n .

Parameters: y : ndarray
The condensed distance matrix.
 $warning$: bool
Invokes a warning if the variable passed is not a valid condensed distance matrix. Explains why the distance matrix is not valid. 'name' is used when referencing the offending variable.
 $throw$: bool
Throws an exception if the variable passed is not a valid condensed distance matrix.
 $name$: string
Used when referencing the offending variable in the warning or exception message.

`scipy.spatial.distance.jaccard(u, v)`

Computes the Jaccard-Needham dissimilarity between two boolean n -vectors u and v , which is

$$\frac{c_{00} + c_{11}}{c_{00} + c_{01} + c_{10} + c_{11}}$$

where c_{ij} is the number of occurrences of $u[k] = i$ and $v[k] = j$ for $k < n$.

Parameters: u : ndarray
An n -dimensional vector.
 v : ndarray
An n -dimensional vector.

Returns: d : double
The Jaccard distance between vectors u and v .

`scipy.spatial.distance.kulsinski(u, v)`

Computes the Kulsinski dissimilarity between two boolean n-vectors u and v , which is defined as

$$rac{c_{TF} + c_{FT} - c_{TT} + nc_{FT} + c_{TF} + n}$$

where c_{ij} is the number of occurrences of $u[k] = i$ and $v[k] = j$ for $k < n$.

Parameters:	u : ndarray An n -dimensional vector.
	v : ndarray An n -dimensional vector.
Returns:	d : double The Kulsinski distance between vectors u and v .

`scipy.spatial.distance.mahalanobis( $u, v, VI$ )`

Computes the Mahalanobis distance between two n-vectors u and v , which is defiend as

$$(u - v)V^{-1}(u - v)^T$$

where VI is the inverse covariance matrix V^{-1} .

Parameters:	u : ndarray An n -dimensional vector.
	v : ndarray An n -dimensional vector.
Returns:	d : double The Mahalanobis distance between vectors u and v .

`scipy.spatial.distance.matching( $u, v$ )`

Computes the Matching dissimilarity between two boolean n-vectors u and v , which is defined as

$$\frac{c_{TF} + c_{FT}}{n}$$

where c_{ij} is the number of occurrences of $u[k] = i$ and $v[k] = j$ for $k < n$.

Parameters:	u : ndarray An n -dimensional vector.
	v : ndarray An n -dimensional vector.
Returns:	d : double The Matching dissimilarity between vectors u and v .

`scipy.spatial.distance.minkowski( $u, v, p$ )`

Computes the Minkowski distance between two vectors u and v , defined as

$$||u - v||_p = (\sum |u_i - v_i|^p)^{1/p}.$$

Parameters:	u : ndarray An n -dimensional vector.
	v : ndarray An n -dimensional vector.
	p : ndarray The norm of the difference $ u - v _p$.
Returns:	d : double The Minkowski distance between vectors u and v .

`scipy.spatial.distance.num_obs_dm( $d$ )`

Returns the number of original observations that correspond to a square, redudant distance matrix D .

Parameters:	<code>d</code> : ndarray The target distance matrix.
Returns:	The number of observations in the redundant distance matrix.

`scipy.spatial.distance.num_obs_y(Y)`
Returns the number of original observations that correspond to a condensed distance matrix `y`.

Parameters:	<code>Y</code> : ndarray The number of original observations in the condensed observation <code>y</code> .
Returns:	<code>n</code> : int The number of observations in the condensed distance matrix passed.

`scipy.spatial.distance.pdist(X, metric='euclidean', p=2, V=None, VI=None)`
Computes the pairwise distances between `m` original observations in `n`-dimensional space. Returns a condensed distance matrix. Each `i` and `j` (where $i < j < n$), the metric `dist(u=X[i], v=X[j])` is computed and stored in the `i,j`th element of the condensed distance matrix.

See `squareform` for information on how to calculate the index of this entry or to convert the condensed distance matrix to a square matrix.

The following are common calling conventions.

1. `Y = pdist(X, 'euclidean')`

Computes the distance between `m` points using Euclidean distance (2-norm) as the distance metric between `m` `n`-dimensional row vectors in the matrix `X`.

2. `Y = pdist(X, 'minkowski', p)`

Computes the distances using the Minkowski distance $\|u - v\|_p$ (p -norm) where $p \geq 1$.

3. `Y = pdist(X, 'cityblock')`

Computes the city block or Manhattan distance between the points.

4. `Y = pdist(X, 'seuclidean', V=None)`

Computes the standardized Euclidean distance. The standardized Euclidean distance between two `n`-dimensional vectors `u` and `v` is

$$\sqrt{\sum (u_i - v_i)^2 / V[x_i]}$$

`V` is the variance vector; `V[i]` is the variance computed over all the `i`'th components of the points. If not passed, it is automatically computed.

5. `Y = pdist(X, 'sqeuclidean')`

Computes the squared Euclidean distance $\|u - v\|_2^2$ between the vectors.

6. `Y = pdist(X, 'cosine')`

Computes the cosine distance between vectors `u` and `v`,

$$\frac{1 - uv^T}{\|u\|_2 \|v\|_2}$$

where $\| \cdot \|_2$ is the 2 norm of its argument.

7. `Y = pdist(X, 'correlation')`

Computes the correlation distance between vectors `u` and `v`. This is

$$\frac{1 - (u - \bar{u})(v - \bar{v})^T}{\|(u - \bar{u})\| \|(v - \bar{v})\|}$$

where $\bar{v}$ is the mean of the elements of vector `v`.

8. `Y = pdist(X, 'hamming')`

Computes the normalized Hamming distance, or the proportion of those vector elements between `u` and `v` that disagree. To save memory, the matrix `x` can be of type boolean.

9. `Y = pdist(X, 'jaccard')`

Computes the Jaccard distance between the points. Given two vectors, u and v , the Jaccard distance between elements $u[i]$ and $v[i]$ that disagree where at least one of them is non-zero.

10. $Y = \text{pdist}(X, \text{'chebyshev'})$

Computes the Chebyshev distance between the points. The Chebyshev distance between two n -maximum norm-1 distance between their respective elements. More precisely, the distance is given by

$$d(u, v) = \max_i |u_i - v_i|.$$

11. $Y = \text{pdist}(X, \text{'canberra'})$

Computes the Canberra distance between the points. The Canberra distance between two points u and v is

$$d(u, v) = \sum_i \frac{|u_i - v_i|}{(|u_i| + |v_i|)}$$

12. $Y = \text{pdist}(X, \text{'braycurtis'})$

Computes the Bray-Curtis distance between the points. The Bray-Curtis distance between two points u and v is

$$d(u, v) = \frac{\sum_i |u_i - v_i|}{\sum_i |u_i| + \sum_i |v_i|}$$

13. $Y = \text{pdist}(X, \text{'mahalanobis'}, \text{VI=None})$

Computes the Mahalanobis distance between the points. The Mahalanobis distance between two points u and v is $\sqrt{(u-v)^T V^{-1} (u-v)}$ where V^{-1} (the `VI` variable) is the inverse covariance. If `VI` is not `None`, it is the inverse covariance matrix.

14. $Y = \text{pdist}(X, \text{'yule'})$

Computes the Yule distance between each pair of boolean vectors. (see yule function documentation)

15. $Y = \text{pdist}(X, \text{'matching'})$

Computes the matching distance between each pair of boolean vectors. (see matching function documentation)

16. $Y = \text{pdist}(X, \text{'dice'})$

Computes the Dice distance between each pair of boolean vectors. (see dice function documentation)

17. $Y = \text{pdist}(X, \text{'kulsinski'})$

Computes the Kulsinski distance between each pair of boolean vectors. (see kulsinski function documentation)

18. $Y = \text{pdist}(X, \text{'rogerstanimoto'})$

Computes the Rogers-Tanimoto distance between each pair of boolean vectors. (see rogerstanimoto function documentation)

19. $Y = \text{pdist}(X, \text{'russellrao'})$

Computes the Russell-Rao distance between each pair of boolean vectors. (see russellrao function documentation)

20. $Y = \text{pdist}(X, \text{'sokalmichener'})$

Computes the Sokal-Michener distance between each pair of boolean vectors. (see sokalmichener function documentation)

21. $Y = \text{pdist}(X, \text{'sokalsneath'})$

Computes the Sokal-Sneath distance between each pair of boolean vectors. (see sokalsneath function documentation)

22. $Y = \text{pdist}(X, \text{'wminkowski'})$

Computes the weighted Minkowski distance between each pair of vectors. (see wminkowski function documentation)

23. $Y = \text{pdist}(X, f)$

Computes the distance between all pairs of vectors in X using the user supplied 2-arity function f . If f is a function that computes the distance between the vectors could be computed as follows:

```
dm = pdist(X, (lambda u, v: np.sqrt(((u-v)*(u-v)).T).sum()))
```

Note that you should avoid passing a reference to one of the distance functions defined in this library.

```
dm = pdist(X, sokalsneath)
```

would calculate the pair-wise distances between the vectors in X using the Python function sokalsneath: sokalsneath being called $\binom{n}{2}$ times, which is inefficient. Instead, the optimized C version is more efficient. The following syntax:

```
dm = pdist(X, 'sokalsneath')
```

Parameters:	X : ndarray An m by n array of m original observations in an n-dimensional space. metric : string or function The distance metric to use. The distance function can be 'braycurtis', 'canberra', 'correlation', 'cosine', 'dice', 'euclidean', 'hamming', 'jaccard', 'kulsinski', 'mahalanobis', 'rogerstanimoto', 'russellrao', 'seuclidean', 'sokalmichener', 'sokalsneath', 'sqeuclidean', 'yule' w : ndarray The weight vector (for weighted Minkowski). p : double The p-norm to apply (for Minkowski, weighted and unweighted) V : ndarray The variance vector (for standardized Euclidean). VI : ndarray The inverse of the covariance matrix (for Mahalanobis).
Returns:	Y : ndarray A condensed distance matrix.
Seealso:	squareform : converts between condensed distance matrices and square distance matrices.

`scipy.spatial.distance.rogerstanimoto(u, v)`

Computes the Rogers-Tanimoto dissimilarity between two boolean n-vectors u and v , which is defined as

$$\frac{R}{c_{TT} + c_{FF} + R}$$

where c_{ij} is the number of occurrences of $u[k] = i$ and $v[k] = j$ for $k < n$ and $R = 2(c_{TF} + c_{FT})$.

Parameters:	u : ndarray An n-dimensional vector. v : ndarray An n-dimensional vector.
Returns:	d : double The Rogers-Tanimoto dissimilarity between vectors u and v.

`scipy.spatial.distance.russellrao(u, v)`

Computes the Russell-Rao dissimilarity between two boolean n-vectors u and v , which is defined as

$$\frac{n - c_{TT}}{n}$$

where c_{ij} is the number of occurrences of $u[k] = i$ and $v[k] = j$ for $k < n$.

Parameters:	u : ndarray An n-dimensional vector. v : ndarray An n-dimensional vector.
Returns:	d : double The Russell-Rao dissimilarity between vectors u and v.

`scipy.spatial.distance.seuclidean(u, v, V)`

Returns the standardized Euclidean distance between two n-vectors u and v . v is an m-dimensional vector usually computed among a larger collection vectors.

Parameters:	u : ndarray An n-dimensional vector. v : ndarray An n-dimensional vector.
--------------------	---

Returns: d : double
The standardized Euclidean distance between vectors u and v .

`scipy.spatial.distance.sokalmichener(u, v)`

Computes the Sokal-Michener dissimilarity between two boolean vectors u and v , which is defined as

$$\frac{2R}{S + 2R}$$

where c_{ij} is the number of occurrences of $u[k] = i$ and $v[k] = j$ for $k < n$, $R = 2 * (c_{TF} + c_{FT})$ and $S =$

Parameters: u : ndarray
An n -dimensional vector.
v : ndarray
An n -dimensional vector.

Returns: d : double
The Sokal-Michener dissimilarity between vectors u and v .

`scipy.spatial.distance.sokalsneath(u, v)`

Computes the Sokal-Sneath dissimilarity between two boolean vectors u and v ,

$$\frac{2R}{c_{TT} + 2R}$$

where c_{ij} is the number of occurrences of $u[k] = i$ and $v[k] = j$ for $k < n$ and $R = 2(c_{TF} + c_{FT})$.

Parameters: u : ndarray
An n -dimensional vector.
v : ndarray
An n -dimensional vector.

Returns: d : double
The Sokal-Sneath dissimilarity between vectors u and v .

`scipy.spatial.distance.sqeuclidean(u, v)`

Computes the squared Euclidean distance between two n -vectors u and v , which is defined as

$$\|u - v\|_2^2.$$

Parameters: u : ndarray
An n -dimensional vector.
v : ndarray
An n -dimensional vector.

Returns: d : double
The squared Euclidean distance between vectors u and v .

`scipy.spatial.distance.squareform(X, force='no', checks=True)`

Converts a vector-form distance vector to a square-form distance matrix, and vice-versa.

Parameters: X : ndarray
Either a condensed or redundant distance matrix.

Returns: Y : ndarray
If a condensed distance matrix is passed, a redundant one is returned, or if a redundant distance matrix is passed, a condensed one is returned.
force : string
As with MATLAB(TM), if force is equal to 'tovector' or 'tomatrix', the input will be treated as a distance vector or distance matrix respectively.
checks : bool
If checks is set to False, no checks will be made for matrix symmetry nor zero diagonal. It is known that $x - x.T$ is small and $\text{diag}(x)$ is close to zero. These values are ignored.

disrupt the squareform transformation.

`scipy.spatial.distance.wminkowski(u, v, p, w)`

Computes the weighted Minkowski distance between two vectors u and v , defined as

$$\left(\sum (w_i |u_i - v_i|^p) \right)^{1/p}.$$

Parameters:

- u : ndarray
An n -dimensional vector.
- v : ndarray
An n -dimensional vector.
- p : ndarray
The norm of the difference $\|u - v\|_p$.
- w : ndarray
The weight vector.

Returns:

- d : double
The Minkowski distance between vectors u and v .

`scipy.spatial.distance.yule(u, v)`

Computes the Yule dissimilarity between two boolean n -vectors u and v , which is defined as

$$\frac{R}{c_{TT} + c_{FF} + \frac{R}{2}}$$

where c_{ij} is the number of occurrences of $u[k] = i$ and $v[k] = j$ for $k < n$ and $R = 2.0 * (c_{TF} + c_{FT})$.

Parameters:

- u : ndarray
An n -dimensional vector.
- v : ndarray
An n -dimensional vector.

Returns:

- d : double
The Yule dissimilarity between vectors u and v .